Here’s the detailed E3 Completion Guide summarizing what was delivered, how the new components work, and how to use and test them.

---

1. Purpose of E3

E3 – LLM Tools & Evaluation Harness adds a deterministic, contract-driven LLM layer atop Slice’s existing thesis and evaluation outputs from E2. The scope:

- **Strictly structured outputs** via Pydantic DTOs (no free text)
- **Pure functions**: no DB, no memory/risk lookups, no side effects
- **LLM prompts with JSON-only response requirement**
- **Reference validation** to catch hallucinated IDs
- **In-memory metrics** for latency/error tracking
- **Diagnostics endpoint** to surface those metrics
- Foundation for E4 so downstream features can rely on structured, safe LLM behavior.

---

2. Key Files and Changes

DTO Models

- `src/slice/models/llm_outputs.py`: `ThesisReview`, `CrossThesisReport`, `IntuitionAnswer`, `DailySummary` (all fields required, no Optional to catch schema drift quickly).

- `src/slice/models/llm_inputs.py`: `Alert`, `DailyContext` (portfolio snapshot, alerts, observations, active theses).

Metrics Infrastructure

- `src/slice/llm/metrics.py`
  - `LLMToolStats` (calls/errors/total latency + `avg_latency_ms` property)
  - `llm_stats`: pre-registered entries for `thesis_review`, `cross_theses`, `intuition_qa`, `daily_summary`
  - `record_llm_call()` and `reset_stats()` (for tests)

LLM Tools & Prompts

- `src/slice/llm/tools.py`
  - `LLMOutputError`, `extract_json()` (find first `{` ..last `}`)
  - `OrchestratorProtocol`
  - Four tool functions:
    - `llm_review_thesis(thesis, evaluation, orchestrator)`
    - `llm_cross_theses(theses, orchestrator)`
    - `llm_query_intuition(question, observations, orchestrator)`
    - `llm_daily_summary(context, orchestrator)`
  - All tools:
    - Build prompts via `slice.llm.prompts`
    - Call `orchestrator.run_session` with *explicit* options:

      ```python
      SessionOptions(
          use_memory=False,
          use_risk=False,
          skip_ingest=True,
      ```

skip_memory=True,

skip_risk=True,

)

```

- Parse JSON (direct parse first, then fallback to `extract_json`)

- Reference validation (see section 4)

- Metrics captured in `try/finally` via `record_llm_call`

- `src/slice/llm/prompts.py`

- Builders for each tool

- Prompts explicitly instruct:

- “Use ONLY the information provided”

- “Respond with a single JSON object matching this schema and nothing else”

- Schema snippets call out reference fields (observation IDs, thesis IDs)

- Contract tests assert sentinel data (thesis titles, question text, etc.) appear in the prompt so DTO-to-prompt wiring is tested.

### Session Options / Orchestrator

- `src/slice/session/models.py`

- Added `skip_ingest`, `skip_memory`, `skip_risk`

- `SessionResponse.observation_id` is now `Optional[int]`

- `src/slice/session/orchestrator.py`

- Honors skip flags: ingestion, memory, risk

- Logging only occurs if ingestion happened

- Tests ensure skip flags override `use_*` even when `True`.
```

Diagnostics API

- `src/slice/api/diagnostics_routes.py`
  - `GET /api/v1/diagnostics/llm` returns:
    ```json
    {
      "llm_tool_metrics": {
        "thesis_review": {"calls": int, "errors": int, "avg_latency_ms": float},
        ...
      }
    }
    ```
- `src/slice/api/main.py` registers the diagnostics router.

---

3. LLM Tool Behavior

Common Flow

1. Prompt builder flattens structured inputs into deterministic instructions.
2. `SessionOptions` explicitly disable all side effects (ingestion, memory, risk).
3. `orchestrator.run_session` returns `SessionResponse.llm_response` (raw string).
4. Response parsing:
   - Direct parse with Pydantic
   - If fails, `extract_json` salvages JSON substring and parse again
   - On failure, raises `LLMOutputError` and metrics record an error
5. Reference validation (see below)

6. Metrics recorded in `finally` (`calls++`, `errors++` when !success)

Reference Validation Details

- `llm_query_intuition`:

  - Allowed IDs from provided `observations`

  - Filter references to allowed IDs, dedupe preserving order

  - If any IDs dropped (hallucinations), set `result.insufficient_context = True`

- `llm_daily_summary`:

  - Allowed IDs from `context.active_theses`

  - Same filtering/deduping logic

- Tests include negative cases where the model set `insufficient_context=False` but included hallucinated IDs; we assert the flag flips to `True`.

No references exist for `ThesisReview` or `CrossThesisReport`, so they're accepted as-is (structure only).

---

4. Metrics & Diagnostics

- `LLMToolStats.calls` increments for every attempt (success or failure).

- `errors` increments only when parse/validation fails.

- `avg_latency_ms` uses all attempts (success + failure) by dividing `total_latency_ms` by `calls`.

- Diagnostics endpoint exposes all metrics so downstream systems (or E4) can monitor reliability and latency.

---

5. Tests & How to Run

Test Suites

- `tests/e3/test_orchestrator_flags.py` – skip flag behavior, including explicit E3-mode tests.

- `tests/e3/test_llm_schemas.py` – DTO validation (missing fields raise `ValidationError`, empty lists allowed).

- `tests/e3/test_metrics.py` – metrics aggregation, error counting.

- `tests/e3/test_extract_json.py` – salvage logic, edge cases (only `{`, only `}`, multiple objects).

- `tests/e3/test_thesis_review.py`, `test_cross_theses.py`, `test_intuition_qa.py`, `test_daily_summary.py`

  - Happy paths, insufficient context, malformed JSON, metrics, prompt wiring assertions.

  - Reference validation tests for intuition/daily summary.

- `tests/e3/test_diagnostics_endpoint.py` – ensures endpoint returns all tool keys and updates after simulated calls.

Running Tests

- Narrow scope (fastest):

  ```

  pytest tests/e3 -q

  ```

- Full regression (recommended before merging):

  ```

  pytest -q
  ```

```

- All tests use in-memory mocks (no actual LLM calls).

---

## 6. Usage & E4 Integration

- E3 provides **pure functions** E4 can call without worrying about state:
  - Inputs: already-constructed domain objects (`Thesis`, `ThesisEvaluationResult`, `DailyContext`, etc.)
  - Outputs: Pydantic DTOs ready for downstream processing or UI display.
- E4/E5 can integrate by:
  - Constructing the appropriate input DTOs
  - Providing an orchestrator implementing `OrchestratorProtocol` (usually wrapping the existing SessionOrchestrator)
  - Consuming `ThesisReview`, `CrossThesisReport`, `IntuitionAnswer`, `DailySummary` as structured responses

Because E3 tools rely on Pydantic models and skip flags, higher layers can trust:

- No hidden ingestion or DB changes
- Reference IDs are validated
- Metrics are tracked for reliability monitoring
- Diagnostics endpoint provides observability for operations

---

## 7. Summary
```

- **E3 delivers a contract-driven LLM layer**: structured outputs, predictable prompts, salvage logic, and metrics.

- **All changes are confined to `src/slice/models`, `src/slice/llm`, `src/slice/session`, and tests**; no DB/memory/risk direct usage in tools.

- **Tests cover schema validation, orchestrator flags, metrics, prompt wiring, reference filtering, and diagnostics**.

- E3 is ready for E4 to build on, ensuring LLM interactions are safe and measurable.